

# Chapter 4

## Boosted Decision Trees and XGBoost

The newly-proposed trigger and complementary offline selection – discussed in chapters 5 and 6, respectively – is based on a machine learning construct called a Boosted Decision Tree (BDT). This ML method provides a robust and relatively simple algorithm for binary classification problems, which works well both with the HLT and with subsequent offline classification and event selection. The theoretical details and structure of BDTs is described in section 4.1. The software package XGBoost (short for eXtreme Gradient Boosting) will be used for both the creation and evaluation of the BDTs employed at the trigger and offline analysis levels. This software is described in more detail in section 4.2.

### 4.1 Tree Boosting Basics

The original description of boosted decision trees and gradient descent boosting is described in "Greedy Function Approximation: A Gradient Boosting Machine" by Jerome H. Friedman [22]. The following section is largely based on this seminal paper, with emphasis placed on the facets of the technique most relevant to this paper.

At its core, a binary classification problem is a function estimation problem; a function must be constructed that maps some input values onto a single output score. The general parlance is to label the input variables for a single event as  $\vec{x} = \{x_1, x_2, \dots, x_n\}$ , where  $n$  is the total number of selected input values. The output, however, is a single score, usually labeled  $y$ . There exists

some theoretical function,  $F^*(\vec{x})$  that directly maps  $\vec{x}$  onto  $y$ . This function cannot be known exactly for problems of moderate complexity, so an estimate of this function,  $\hat{F}(\vec{x})$ , is the goal of the gradient boosting process.

To quantify how closely the approximation  $\hat{F}(\vec{x})$  follows the true  $F^*(\vec{x})$ , some loss function must be defined. There are many forms of loss function, as will be described in section 4.2.1. The loss function is termed  $L(y, F(\vec{x}))$ , meaning that it quantifies how closely the approximate points,  $F(\vec{x})$ , follow the true classification values of the events,  $y$ . Given these definitions, the goal of the BDT algorithm can be written as given by Friedman [22].

$$F^* = \min_F E_{y, \vec{x}}[L(y, F(\vec{x}))] \quad (4.1)$$

Functional approximation is a rich and complex field with many possible approaches. So-called "additive" approaches often tend to be both conceptually and computationally simple. In this approach, the total mapping function can be built iteratively as a sum of many simpler functions with well-understood parameters. These functions can be combined as a linear combination with a weight,  $\beta$ , applied to each of the simple functions. The generic functions being added together, in the case of a boosted decision tree, is an individual regression tree [22]. Terming these generic regression functions as  $h(\vec{x}, \vec{a}_m)$ , where  $\vec{a}_m$  are the chosen parameters for the individual regression tree allows the function defined in 4.1 to be rewritten as given in equation 4.2.

$$F(\vec{x}; \{\beta_m, \vec{a}_m\}) = \sum_{m=1}^M \beta_m h(\vec{x}; \vec{a}_m) \quad (4.2)$$

So, then, the goal of the BDT algorithm is no longer to find a functional form that best approximates the mapping function between the inputs and an output score. Instead, with the functional form of  $h$  selected as a regression tree, the parameters for each tree,  $\vec{a}_m$ , and the weights used in adding the trees together,  $\beta_m$ , must be optimized. This additive approach

underpins the methods used by the software package XGBoost, described in the next section.

## 4.2 XGBoost Software and Binary Classification

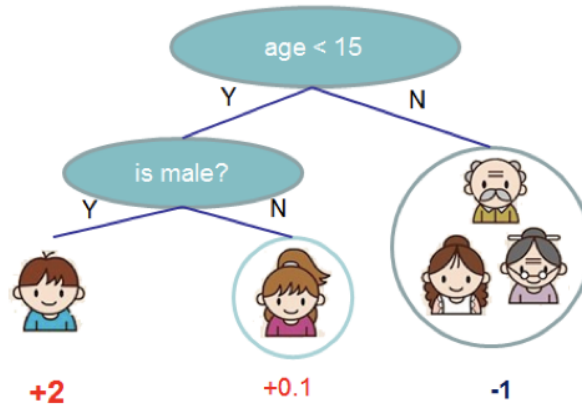
XGBoost is a standard software package for ML methods. Specifically, an HLT must generate a decision on whether to keep or discard an event being evaluated. As a result, triggering is a binary classification problem. The goal, then, is to map the input features for each event,  $\vec{x}_i$ , onto a label for each event,  $y_i$ . For a binary classification problem, these  $y_i$  values are a label of 0 or 1, denoting the two possible classes (in the case of triggering, these cases are 0 for background and 1 for signal). To arrive at a final yes or no decision on each event, the BDT must generate a prediction,  $\hat{y}_i$ , for each event given the input features  $\vec{x}_i$ . Within XGBoost, this prediction can be given as a score between 0 and 1. This score can be understood as the probability that the chosen event is in the signal class. A value of 0 means a predicted 0% probability of the event being signal (thereby meaning the event is certainly background), whereas a score of 1 means a predicted 100% probability of the event being signal. In practice, most scores will lie somewhere between these two extremes. As a result, some threshold can be placed on these output scores to separate signal and background. So, even though XGBoost will output a continuous score between 0 and 1, this threshold (called a score cut) converts this continuous variable into a binary decision on whether to keep or discard an event.

As with the description in the previous section, XGBoost uses a linear combination of so-called decision trees. The full space of possible decision trees is often referred to as the set of classification and regression trees (CART) [22]. Individual instances of functions within the CART can be denoted  $f_k(\vec{x})$ , where  $k$  ranges the number of functions employed, and the vector  $\vec{x}$  still contains all of the input variables chosen for the model. The output decision scores can then be mathematically expressed as given in equation 4.3 (note the similarity to equation 4.2). Of note is the fact that this value is not explicitly normalized; in fact, these predictions may sum to any arbitrary positive or negative value [17]. Normalization and transformation of these scores

into a probability must be done to provide the appropriate output, which is handled by XGBoost and the SciKit-Learn package.

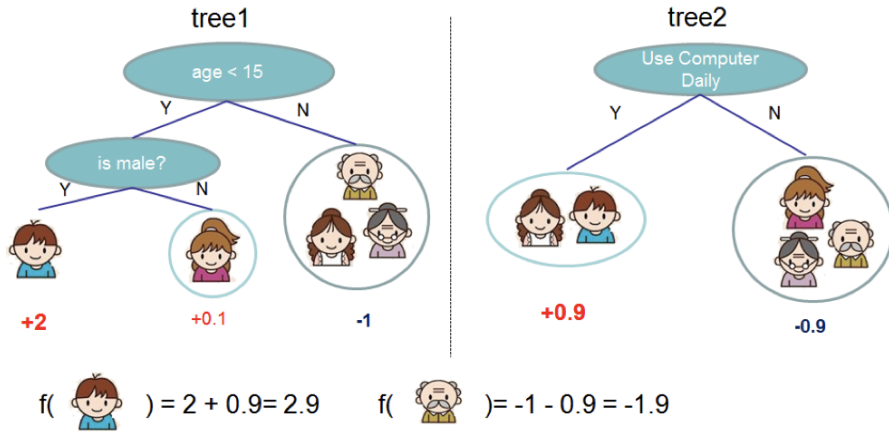
$$\hat{y}_i = \sum_{k=1}^K f_k(\vec{x}_i) \quad (4.3)$$

While a very simple task may be accomplished with a single decision tree, a complex model can be constructed with what is known as a tree ensemble. In order to understand this linear combination of decision trees, of course, the individual decision tree must first be understood. A very basic schematic approach can first be taken. A decision tree is so called due to its branching structure. Essentially, a decision tree functions like a flow chart. Each "node" or "leaf" of the tree is essentially a question with a yes or no answer. If the answer is yes, the chart splits in one direction to the next node whereas an answer of no splits the chart to a different node. This branching structure allows relatively complex decisions to be made with relatively simple individual nodes. The more layers of nodes are nested together, the more complex the model becomes. In figure 4.1, a very simple schematic of a two-layer decision tree can be seen. In this schematic, the input features,  $\vec{x}_i$ , are probed by each node. The features for this tree could be denoted as  $\vec{x}_i == \{\text{age, gender}\}$ .



**Figure 4.1.** A very simple schematic of a single decision tree. Each question, called a node, employs an individual feature of the input data to determine which of the next nodes should be traversed. The weights applied by the nodes can be seen under each classification group. Reproduced from [4].

This schematic shows not only the basic structure of a decision tree, but also the mathematical weights assigned by the decision tree. So, then, by using the inputs  $\vec{x}_i$ , the score of the tree can then be given by  $y_{i,k} = f_k(\vec{x}_i)$  as in equation 4.3. Of course, the ensemble will consist of a number of such trees, such that the total weight will be assigned by the linear combination of individual tree weights. Schematically, this can be shown in figure 4.2, which has been reproduced from [4].



**Figure 4.2.** A very simple schematic of an ensemble of two decision trees. Each tree assigns a weight given by  $f_k \vec{x}_i$ , and these weights can be summed for each input member,  $i$ , to render the final event score. Reproduced from [4].

Of course, the determination of the weights at each node must be optimized. In addition, the number of trees and the overall complexity of these trees must be quantified and optimized for the specific problem at hand. This leads to the implementation of an objective function, described in the next section.

### 4.2.1 The Objective Function

While the loss function alone, employed in equation 4.1, is a good starting point for function approximation, it does not capture all the necessary complexity of a software-based algorithm. Instead, a so-called objective function can be constructed [4] [17]. The objective function begins from a chosen loss function as before. An additional term, called a regularization term, is added to the objective function to penalize overly-complex models. This works toward

two aims: so-called overfitting must be avoided, as finding a final functional form that too closely fits the input data reduces the generalizability of the model; the penalization of model complexity further works to ensure that the software selects the simplest possible tree structure to achieve the desired accuracy. Since ML algorithms are, in practice, deployed on finite computational resources, penalizing overly complex models is necessary.

$$obj(\vec{\theta}) = L(\vec{\theta}) + \Omega(\vec{\theta}) \quad (4.4)$$

The new objective function can then be written schematically as shown in equation 4.4 [4]. Here,  $L(\vec{\theta})$  is a loss function, and  $\Omega(\vec{\theta})$  is the regularization term. While the loss function concerns itself with individual data points, the regularization term concerns itself with each decision tree in the ensemble. So, The loss function will be summed over all data points, whereas the regularization will be summed over each tree in the ensemble. If we take the functions within the CART space,  $f_k(\vec{x}_i)$  as in equation 4.3, we can then express the overall objective function more concretely as equation 4.5 [17].

$$obj(\vec{\theta}) = \sum_{i=1}^n L(y_i, \hat{y}_i^{(t)}) + \sum_{k=1}^t \omega(f_k) \quad (4.5)$$

Where n is the total number of data points and t is the total number of individual trees in the ensemble. In this new formulation,  $\omega f_k$  is referred to as the complexity score of the model – simpler decision trees will have a lower complexity score. While a more complex model will likely lower the loss function, it will also raise the complexity score of the trees and therefore may not necessarily minimize the objective function. This objective function, then, must be minimized; the inclusion of both the loss and regularization term allows the optimal balance between accuracy and complexity to be reached.

In equation 4.5, one can notice that the prediction has been given a superscript (t). This prediction,  $\hat{y}_i^{(t)}$  denotes the predicted value at the  $t^{th}$  step in the training process. In XGBoost, each step t introduces one additional decision tree to the ensemble. This additive process leads

to two different avenues to control the complexity of the model; each individual decision tree may have fewer nodes, or the total number of trees may be reduced. The prediction score at each step can then be built up iteration by iteration. A starting point is chosen, then each additional tree added to the ensemble seeks to further refine the model. Of course, the parameters of each tree,  $\vec{\theta}$ , can be optimized at each step to minimize the objective function. This additive approach is shown iteratively in equation 4.6, where the connection to the previous definition in equation 4.3 is obvious.

$$\begin{aligned}
\hat{y}_i^{(0)} &= 0 \\
\hat{y}_i^{(1)} &= f_1(\vec{x}_i) = \hat{y}_i^{(0)} + f_1(\vec{x}_i) \\
\hat{y}_i^{(2)} &= f_1(\vec{x}_i) + f_2(\vec{x}_i) = \hat{y}_i^{(1)} + f_2(\vec{x}_i) \\
&\dots \\
\hat{y}_i^{(t)} &= \sum_k^t f_k(\vec{x}_i) = \hat{y}_i^{(t-1)} + f_t(\vec{x}_i)
\end{aligned} \tag{4.6}$$

Combining this with the definition of the objective function in equation 4.5 gets to the heart of the XGBoost boosted decision tree algorithm; at each iterative step, a new tree must be added that seeks to further minimize the objective function. To best elucidate the iterative construction of an optimized tree, the objective function at each step must be quantified. The definition of the  $t^{th}$  prediction given in equation 4.6 can then be added to the definition of the objective function given in equation 4.5. This yields the expression in equation 4.7.

$$obj(\vec{\theta})^{(t)} = \sum_{i=1}^n L(y_i, \hat{y}_i^{(t-1)} + f_t(\vec{x}_i)) + \sum_{k=1}^t \omega(f_k) \tag{4.7}$$

(ASK! How can we reduce the sum over  $\omega(f_k)$  to just  $\omega(f_t) + \text{constant}??$ )

Now, of course, the loss and regularization terms must be defined and understood such that the objective at each step can be concretely defined in terms of the model parameters,  $\vec{\theta}$ .

For the case of a logistic loss, an analytical form of the iterative loss function is rather complex, especially if a summation over it is desired. Instead, the Taylor expansion of the loss function can be employed. It is generally a sound procedure to only keep up to the quadratic term in the Taylor expansion [4]. The expansion up to the quadratic term can be seen in equation 4.8. As they will be used in subsequent steps, the first and second derivative terms are defined as  $g_i$  and  $h_i$ , respectively (as seen in equation 4.9).

$$obj(\vec{\theta})^{(t)} = \sum_{i=1}^n [L(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(\vec{x}_i) + \frac{1}{2} h_i f_t^2(\vec{x}_i)] + \sum_{k=1}^t \omega(f_k) \quad (4.8)$$

$$\begin{aligned} g_i &= \partial_{\hat{y}_i^{(t-1)}} L(y_i, \hat{y}_i^{(t-1)}) \\ h_i &= \partial_{\hat{y}_i^{(t-1)}}^2 L(y_i, \hat{y}_i^{(t-1)}) \end{aligned} \quad (4.9)$$

This general form allows for any arbitrary loss function to be optimized; as long as the change in the loss function can be quantified, the constants  $g_i$  and  $h_i$  can be determined and the objective can be updated at that step. This powerful formulation allows XGBoost to calculate an updated objective relatively rapidly for each new tree added to the ensemble, which therefore allows the training process to be closely monitored.

Before the final objective function can be quantified, the regularization term must be understood. The complexity functions,  $\omega_k(f)$ , must be expanded, then. In order to do this, an understanding of the form of the individual tree functions,  $f_t(\vec{x})$ , must be analyzed. As shown in the simplified schematic in figure 4.2, these functions are a collection of weights assigned by each node in the tree. While the assignment of these weights is relatively complex, the tree itself is a simple addition of the weights. Given some function that assigns the weights at each node of the tree,  $q(\vec{x})$ , the tree function can be defined as in equation 4.10. This  $q(\vec{x})$  is usually called the tree structure function, as it determines the splitting of the tree into different leaves and how the weights are assigned at each leaf.



$$f_t(\vec{x}) = w_{q(\vec{x})} \quad (4.10)$$

If we take  $T$  to be the total number of leaves in the individual tree, then the weights will be  $T$  real numbers. Larger weights lead to larger changes in the total output score, so the tree should generally try to reduce the swings in output by restricting these weights to be lower. It then becomes clear that there are two sources for complexity within a tree ensemble that must be controlled – the total number of leaves increases the complexity, as does the total sum of weights. This suggests the form of the complexity function given in equation 4.11.

$$\omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 \quad (4.11)$$

These two new values,  $\gamma$  and  $\lambda$ , are known as the L1 and L2 regularization terms. These tunable parameters are usually considered hyperparameters to distinguish them from the other parameters in the model; while parameters are adjusted at each step to minimize the objective function, hyperparameters are fixed before the training begins as a constant throughout the training processes and apply to all trees equivalently. There are additional hyperparameters to an XGBoost BDT model, which will be discussed further in chapter 5.

With the updated form of  $f_t(\vec{x})$  and  $\omega(f)$ , the full objective function at a given step can be written as equation 4.12 [4].

$$obj(\vec{\theta})^{(t)} \approx \sum_{i=1}^n [g_i w_{q(\vec{x})} + \frac{1}{2} h_i w_{q(\vec{x})}^2] + \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 \quad (4.12)$$

Immediately, it can be seen that there is a sum over the index  $j$  runs over all of the leaves in the tree from  $j=0$  to  $T$ , whereas the sum over  $n$  sums all of the data points at each leaf,  $j$ . This can then cleverly be written into an easier to digest form, where terms with the same powers of  $w_j$  can be grouped. The set  $I_j = \{i | q(\vec{x}) = j\}$  can then be defined as the set of indices where the data point is in the specific leaf,  $j$ . This makes sense, as each leaf in the tree will not use all input

data points. Returning to the example in figure 4.2, it is clear that three of the data points are split at the first leaf, leaving only two that make it to the second leaf. So, then, the data points that are not involved in the second leaf need not be summed over for that leaf when determining the objective function. This means that the final objective function can be rewritten as a sum over all leaves,  $j$ , as given in equation 4.13.

$$obj(\vec{\theta})^{(t)} \approx \sum_{j=1}^T [(\sum_{i \in I_j} g_i)w_j + \frac{1}{2}(\sum_{i \in I_j} h_i + \lambda)w_j^2] + \gamma T \quad (4.13)$$

Finally, these sums over the index  $i$  can be defined as their own value for simplicity. Since for each  $j$  there will be one value of this sum over  $i$ , the quantities  $G_j = \sum_{i \in I_j} g_i$  and  $H_j = \sum_{i \in I_j} h_i$  can be defined. This simplification of notation allows the final form of the objective function to be written as in equation 4.14.

$$obj(\vec{\theta})^{(t)} \approx \sum_{j=1}^T [G_j w_j + \frac{1}{2}(H_j + \lambda)w_j^2] + \gamma T \quad (4.14)$$

Differentiating this with respect to  $w_j$  and finding where it is 0 allows the individual leaf weight that minimizes the objective function to be found as given in equation 4.15. Then, these weight can be put into the objective function to find the maximum reduction in objection function for a single tree iteration,  $t$ . This is shown in equation 4.16.

$$\omega_j^* = -\frac{G_j}{H_j + \lambda} \quad (4.15)$$

$$obj^* = -\frac{1}{2} \sum_{j=1}^T \frac{G_j^2}{H_j + \lambda} + \gamma T \quad (4.16)$$

The form of this equation makes the logic of the minimization process clear. The first term depends on  $H_j$  and  $G_j$ , which are determined by the rate of change of the loss function. So, an update that more rapidly changes the loss function will more rapidly reduce the objective function given that this term is negative. The second term simply shows that making a more

complex tree with more leaves will increase the objective function. A balance, therefore, will be specifically found by the algorithm that helps maximally reduce the loss at each step while still ensuring that the complexity of the model is kept relatively low.

### 4.2.2 Tree Boosting Algorithm

So, with equations 4.15 and 4.16, there exists a way to evaluate how well-optimized a specific tree structure,  $q(\vec{x})$ , is. How, though, are the possible structures explored? While the above method would work well if all structures could be evaluated and the optimum structure selected, this is not at all practical in a computational sense. Given that BDTs can be trained on hundreds of thousands to millions of events with dozen of input variables and possibly thousands of trees in the ensemble, an efficient approach to additive tree boosting must be found.

The most straightforward algorithm is called the exact greedy algorithm [4]. This algorithm begins with one single leaf, then iterates over all possible splits at that leaf. This process is repeated until either a maximum depth is reached, or additional splits no longer improve the objective function. The issue, therefore, changes from trying to directly find the optimum tree structure to simply evaluating the possible splits at each node, choosing the optimum split, then iterating. While equation 4.16 is useful to evaluate an entire tree, it does not directly help with the splitting process. Instead, a new form of the objective function must be determined in terms of the proposed split.

Starting from the general objective score found in equation 4.16, a split can be defined. Since the chosen tree structure is a binary decision tree, there are only ever two possible branches created at each node. These branches can be termed left and right, for convenience. Before, the set  $I_j = \{i | q(\vec{x}) = j\}$  was defined such that all data points  $y_i$  within a particular node were contained within it. A similar set can then be defined for the left and right side of side of the split, denoted by  $I_L$  and  $I_R$ , respectively. Then, the terms  $G_j$  and  $H_j$  can be redefined in terms of these sets for a single value of  $j$  as given in equation 4.17.

$$\begin{aligned}
G_L &= \sum_{i \in I_L} g_i & G_R &= \sum_{i \in I_R} g_i \\
H_L &= \sum_{i \in I_L} h_i & H_R &= \sum_{i \in I_R} h_i
\end{aligned} \tag{4.17}$$

This allows the split objective function to be written as in equation 4.18 [4]. Of note is the fact that this is the objective contribution from a single split, so  $T = 1$  in this case. Additionally, the terms  $G_I$  and  $H_I$  are taken to be the sums over the entire set  $I$  before the split.

$$obj_{split} = \frac{1}{2} \left[ \frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G_I^2}{H_I + \lambda} \right] + \gamma \tag{4.18}$$

In practice, this form of the objective function is actually employed by the XGBoost algorithm. Using this minimization process, individual leaves can be added to each tree such that the gain in accuracy and the gain in complexity are adequately balanced for each additional split added.

There are two additional ways in which overfitting (i.e. overcomplexity) of the model is avoided. One applies to the individual weight values, whereas the other applies to each tree. Shrinkage was first described by Friedman, and is a factor (denoted by  $\eta$  in XGBoost) applied to every weight within every tree [22]. This shrinkage aims to slow down the learning process, allowing the contribution of each tree to be controlled and therefore allowing models with more individual trees that each have a smaller impact to avoid overfitting. The learning rate,  $\eta$ , is another hyperparameter of the model in XGBoost – it is tuned once for each model. The effects of the learning rate will be described in more detail in chapter 5.

The other way to avoid overfitting is to restrict the set of input features used by each tree. Taking a simple vector of  $d$  input features,  $\vec{x}_i = \{x_1, x_2, x_3, \dots, x_d\}$ , this vector can be sampled differently for each tree in an ensemble. So, where the first tree in the ensemble may take its input features as  $x^{(1)}_i = \{x_1, x_2, x_3\}$ , the second tree may take  $x^{(2)}_i = \{x_{d-2}, x_{d-1}, x_d\}$ . This further

limits the power of each individual tree and the redundancy of each tree in the ensemble, thereby lowering the chance of overfitting. The proportion of all available input features that should be taken at each tree is an additional hyperparameter that can be set for the whole ensemble.